

Dynamic Pricing for Airbnb Listings



Next Slide



EDA INSIGHT

- Task: Predict listing price (Regression)
- Data: European Airbnb listings (Aemf1)
- Size: ~41,000 listings
- Target: Price



```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 41714 entries, 0 to 41713
Data columns (total 19 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   City                                  41714 non-null  object
1   Price                                41714 non-null  float64
2   Day                                  41714 non-null  object
3   Room Type                            41714 non-null  object
4   Shared Room                          41714 non-null  bool
5   Private Room                         41714 non-null  bool
6   Person Capacity                      41714 non-null  float64
7   Superhost                            41714 non-null  bool
8   Multiple Rooms                       41714 non-null  int64
9   Business                             41714 non-null  int64
10  Cleanliness Rating                   41714 non-null  float64
11  Guest Satisfaction                   41714 non-null  float64
12  Bedrooms                             41714 non-null  int64
13  City Center (km)                     41714 non-null  float64
14  Metro Distance (km)                  41714 non-null  float64
15  Attraction Index                     41714 non-null  float64
16  Normalised Attraction Index          41714 non-null  float64
17  Restraunt Index                      41714 non-null  float64
18  Normalised Restraunt Index            41714 non-null  float64
dtypes: bool(3), float64(10), int64(3), object(3)
memory usage: 5.2+ MB
```

```
df = df.dropna()
```

Before using
dataset and
analyze we
cleaned,deleted
NaN values



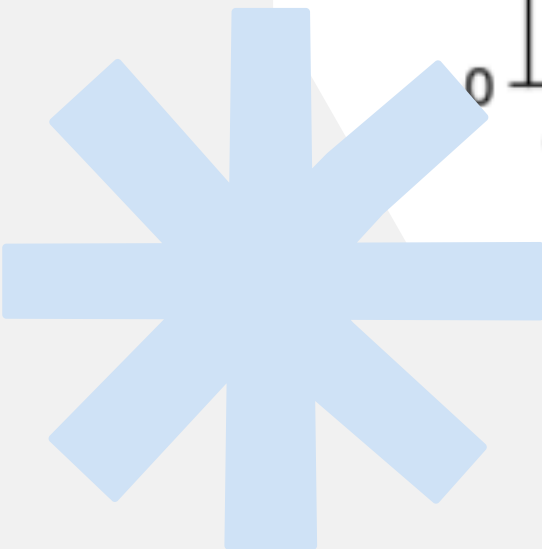


Price Distribution (Right- Skewed)

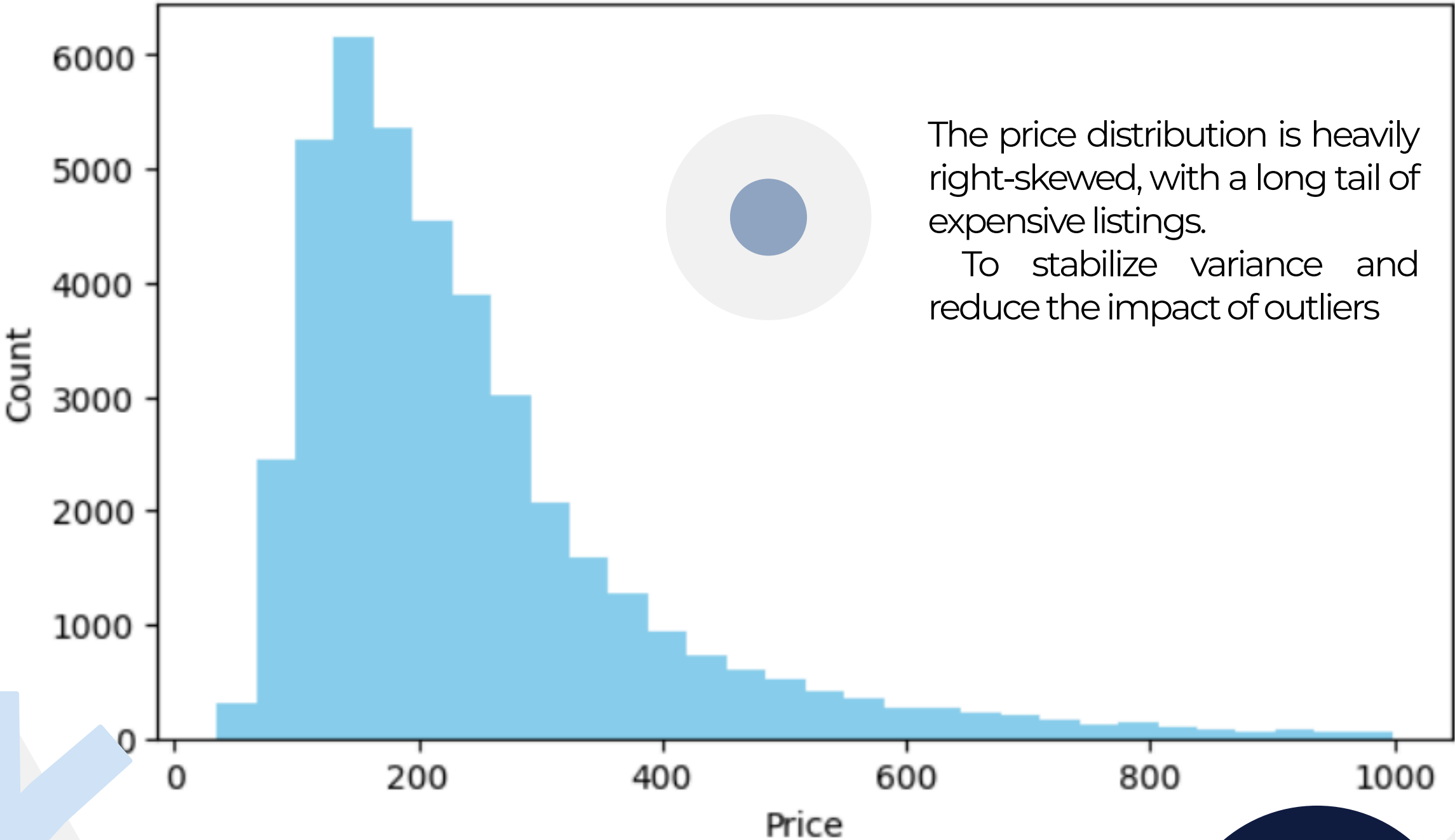
Next Slide



PRICE BY ROOM TYPE



Distribution of Airbnb Prices





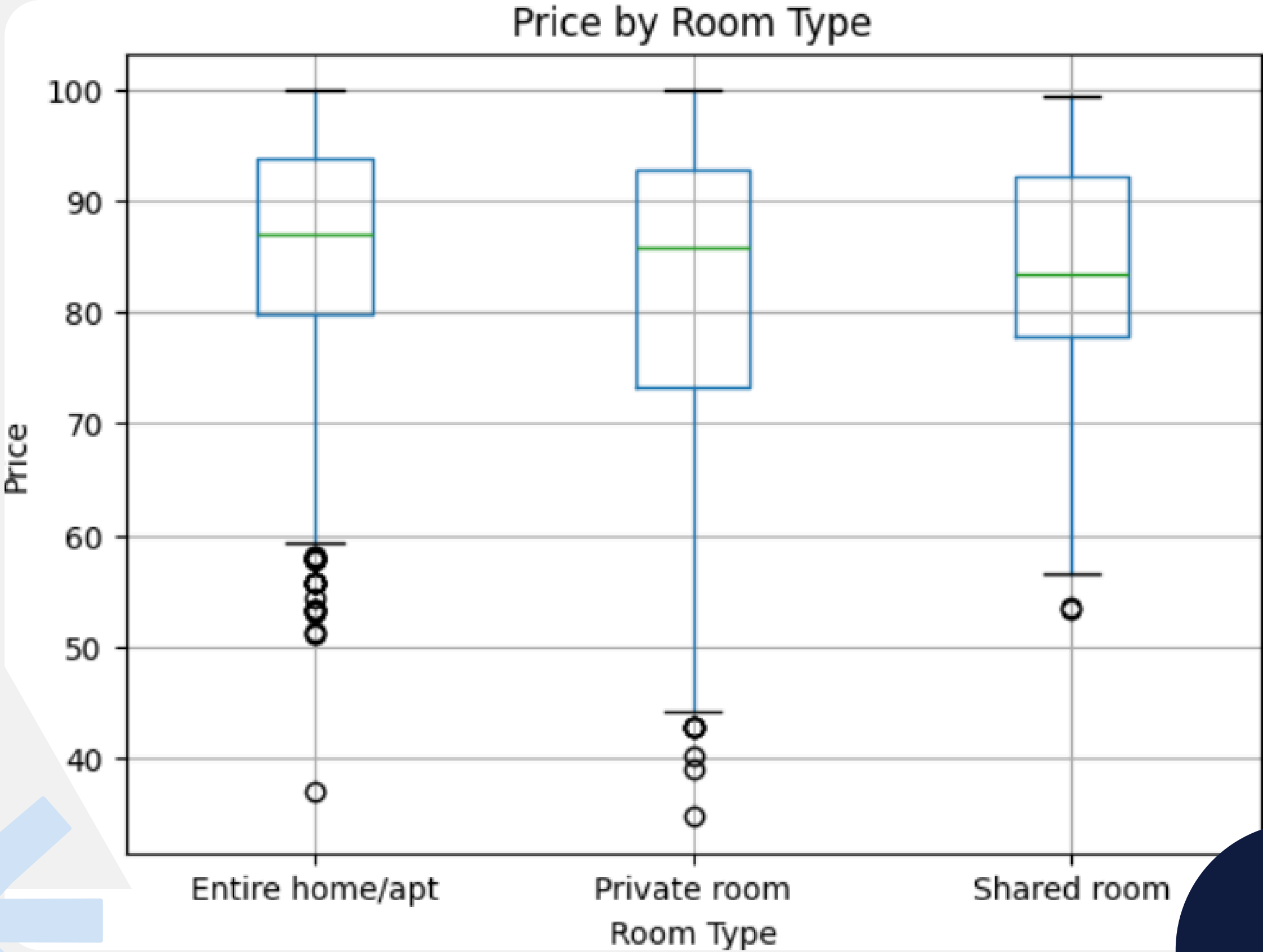
Price by Room Type

Next Slide



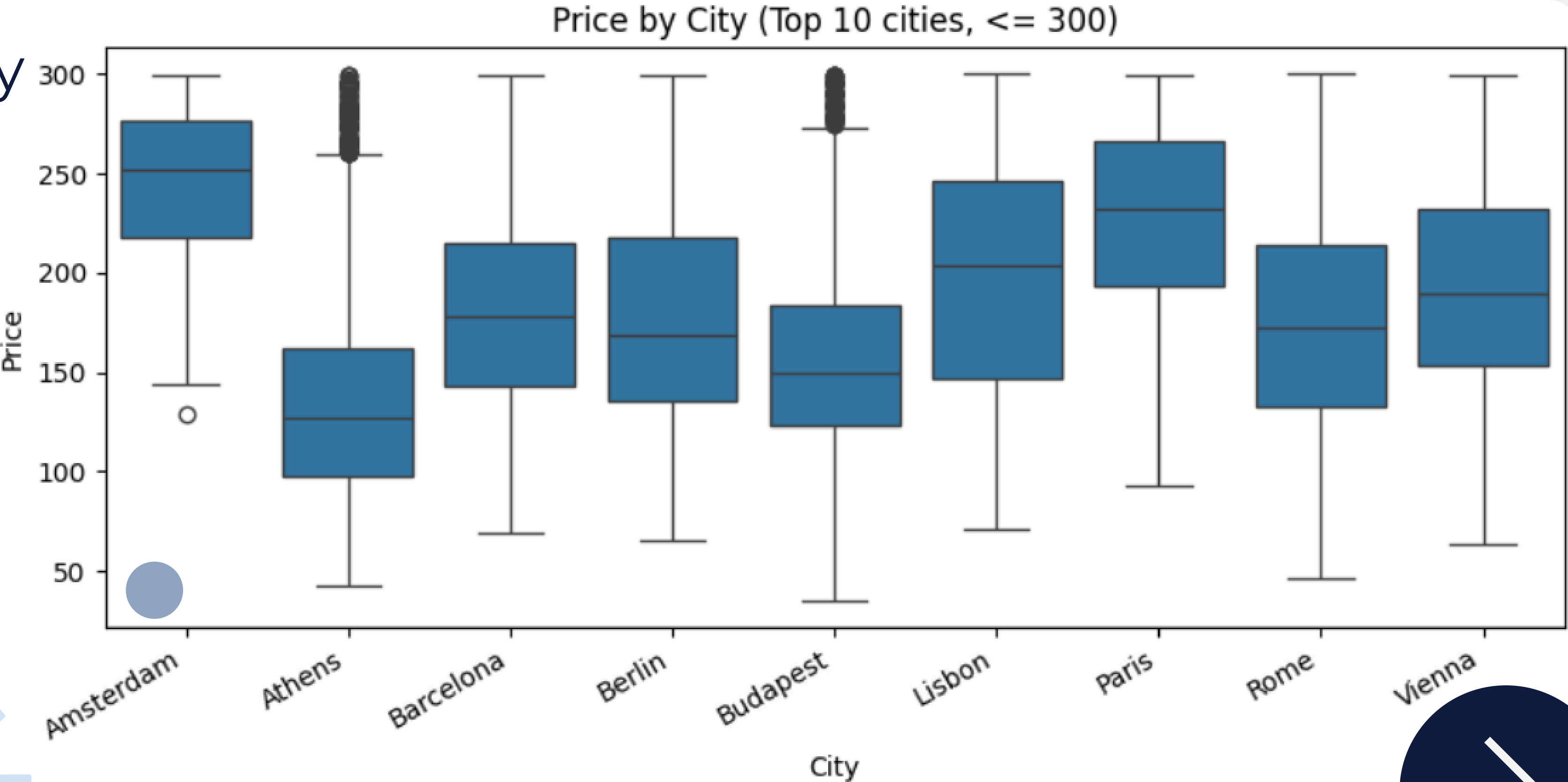
FEATURES
PREPROCESSING

&





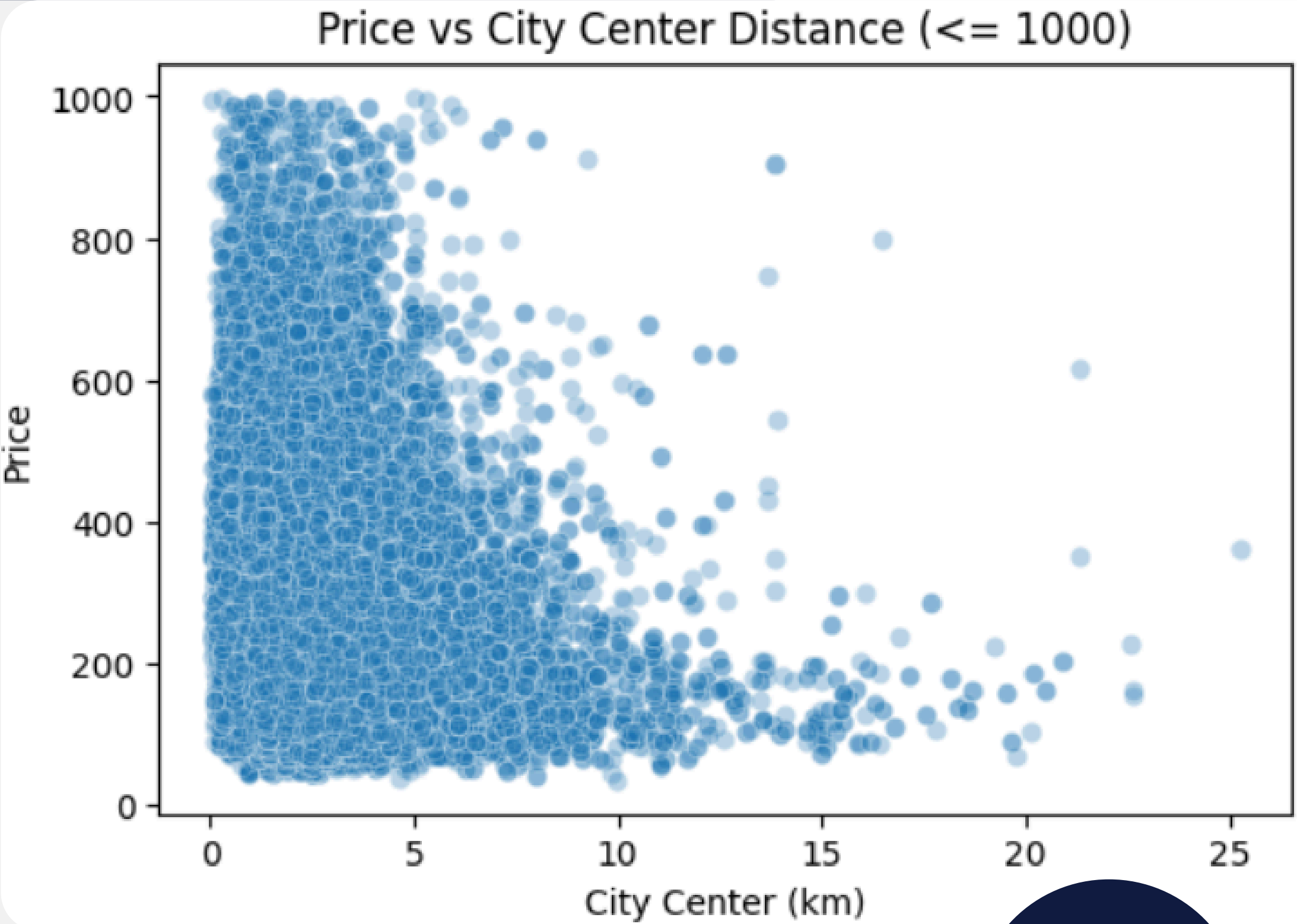
Price by City





Price by City center distance

A SCATTER PLOT WAS USED TO ANALYZE THE RELATIONSHIP BETWEEN AIRBNB PRICE AND DISTANCE TO THE CITY CENTER. THE RESULTS SHOW A CLEAR TREND: LISTINGS CLOSER TO THE CITY CENTER TEND TO HAVE HIGHER PRICES, WHILE PRICES GENERALLY DECREASE AS THE DISTANCE INCREASES.



Next Slide

CORRELATION MATRIX



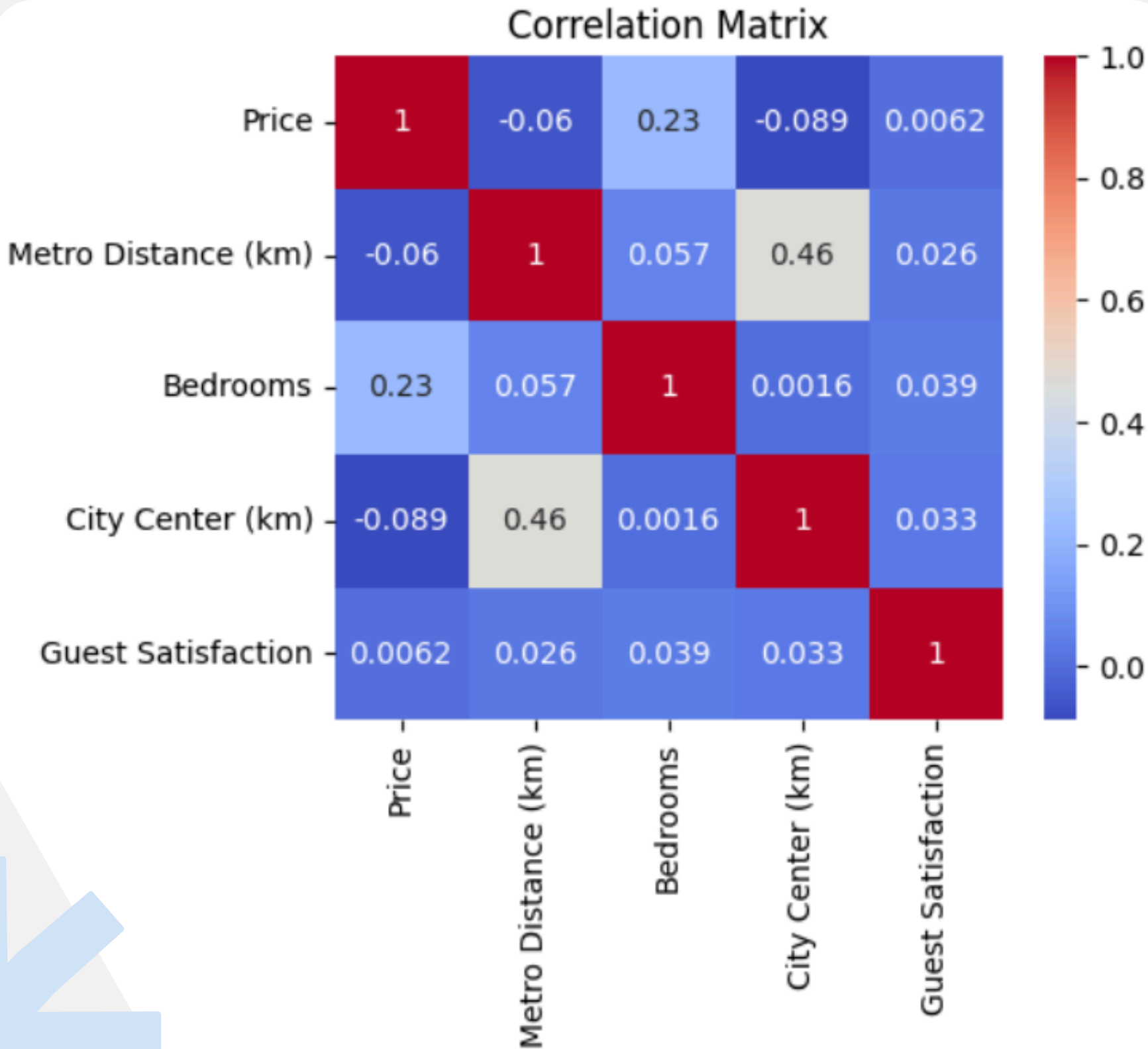


Correlation Matrix

Next Slide

FEATURES
PREPROCESSING

&





Features Preprocessing

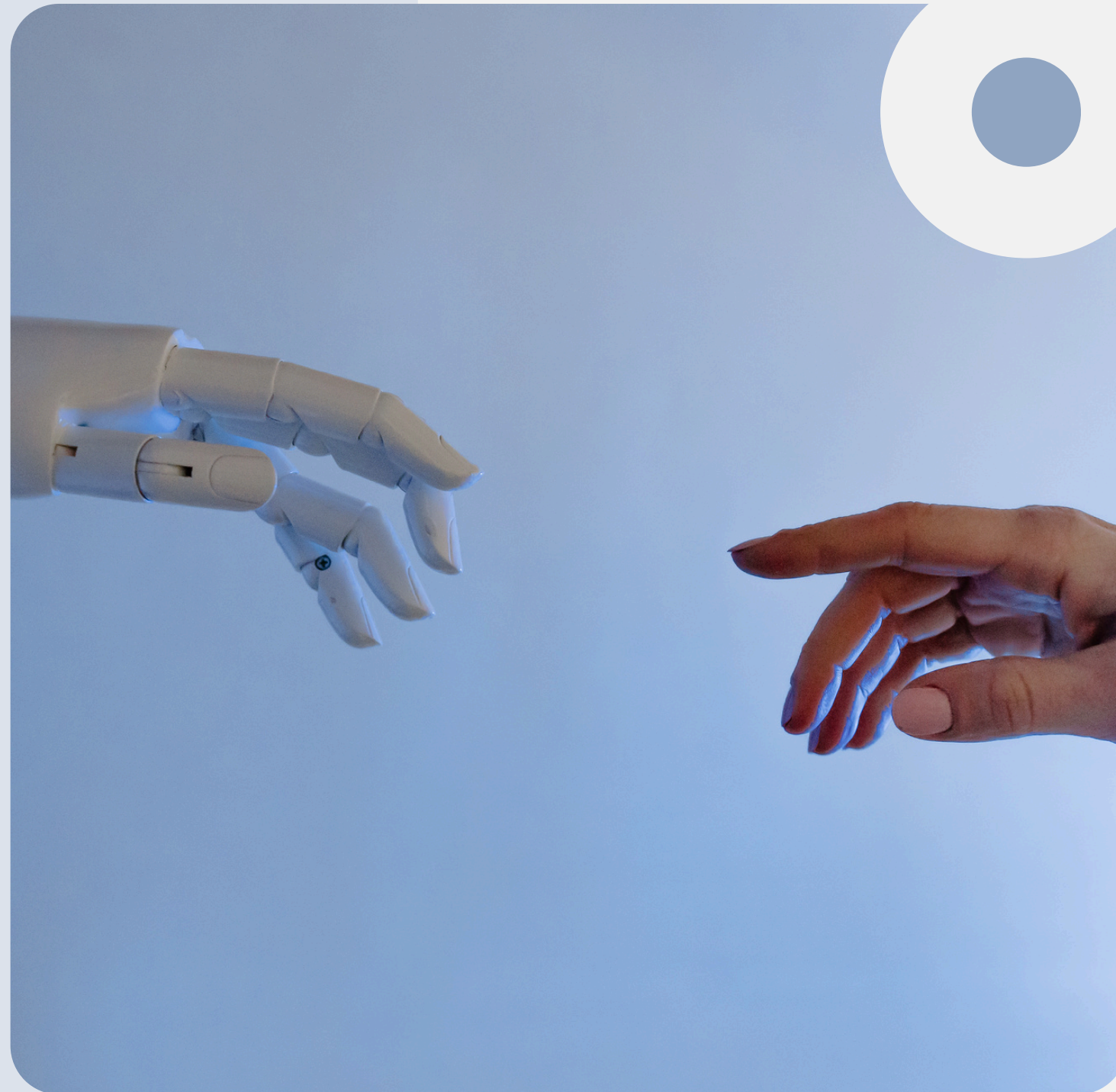
- Categorical → One-hot encoding
- Numerical → kept as is
- Train / Test split: 80 / 20

ALL REMAINING COLUMNS WERE USED AS INPUT FEATURES.

CATEGORICAL VARIABLES WERE CONVERTED INTO NUMERICAL FORM USING ONE-HOT ENCODING TO MAKE THEM SUITABLE FOR MACHINE LEARNING MODELS.

Next Slide





Models

Champion vs Challenger

We used a Champion–Challenger approach.

Linear Regression was chosen as an interpretable baseline model.

XGBoost was selected as a non-linear challenger to capture complex relationships in pricing.





Results & Metrics

	Model	RMSE	R ²
0	Linear Regression	277.874413	0.670453
1	XGBoost	270.291689	0.748688

Next Slide



FINAL SOLUTION & LIMITATIONS



RMSE is reported on the original price scale for business interpretability.

R² is calculated on the log-transformed target, since the model was trained in that space.

XGBoost achieved the lowest RMSE and the highest R² after hyperparameter tuning.



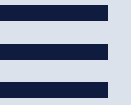


```
: param_grid = {  
    "max_depth": [3, 5],  
    "n_estimators": [200, 400],  
    "learning_rate": [0.05, 0.1],  
    "subsample": [0.8, 1.0],  
    "colsample_bytree": [0.8, 1.0],  
}  
  
grid = GridSearchCV(  
    XGBRegressor(objective="reg:squarederror", random_state=42),  
    param_grid,  
    cv=3,  
    scoring="neg_root_mean_squared_error",  
    n_jobs=-1  
)  
  
grid.fit(X_train, y_train)  
best_model = grid.best_estimator_  
  
pred_best_log = best_model.predict(X_test)  
pred_best_real = np.expm1(pred_best_log)  
  
best_rmse = np.sqrt(mean_squared_error(y_test_real, pred_best_real))  
best_r2 = r2_score(y_test, pred_best_log)  
  
(best_rmse, best_r2, best_model.__class__.__name__)  
  
: (np.float64(266.3358402211994), 0.7575653649509989, 'XGBRegressor')
```

Hyperparameter Tuning GridSearchCV

- Applied GridSearchCV to optimize key parameters: tree depth, number of estimators, learning rate, and subsampling.
- Evaluated model with RMSE and R^2 on test data.





```
predict_price(  
    city="Amsterdam",  
    day="Weekday",  
    room_type="Private room",  
    person_capacity=2,  
    superhost=0,  
    multiple_rooms=0,  
    business=0,  
    cleanliness_rating=9,  
    guest_satisfaction=95,  
    bedrooms=1,  
    city_center_km=2,  
    metro_distance_km=0.5,  
    attraction_index=100,  
    normalised_attraction_index=30,  
    restraunt_index=500,  
    normalised_restraunt_index=50,  
    shared_room=0,  
    private_room=1  
)
```

712.848388671875

Final Solution & Limitations

The final notebook ends with a prediction function that takes raw listing inputs and outputs a predicted price.

Limitations include city-level bias and the lack of seasonality or event-based effects.

Therefore, the model represents an average expected price rather than real-time dynamic pricing.

PREDICTION FUNCTION
IMPLEMENTED

TAKES RAW INPUT
→ OUTPUTS PRICE

LIMITATIONS: CITY
BIAS, NO
SEASONALITY